Coding & Learning: Using GenAI To Support Software Development

Course Exercises with Suggested Prompts

This document lists all **20 exercises** from the WeThinkCode_ *AI Software* course (ai.wethinkco.de/ai-software). Each exercise is reproduced in full, followed by a **Suggested prompt** you can adapt and paste into your AI tool (Claude / GitHub Copilot chat) for that specific exercise.

Module 2 — Using AI for Everyday Work

Chapter: Using AI to comprehend existing codebases

Exercise 1 — Knowing where to start

In this exercise, you'll practice using AI prompts to understand the Task Manager codebase as if it were an unfamiliar project you've just encountered. You'll apply the three prompt scenarios to build your understanding of the project structure, locate specific features, and understand the domain model.

Setup

Choose the Task Manager implementation in your preferred language (Python, JavaScript, or Java) from this gitlab repo.

Create a new document to track your discoveries and understanding during this exercise.

Imagine you have just joined a team responsible for maintaining this Task Manager application, and you need to understand it quickly to start contributing.

Check out the starter code at this Gitlab Repo. For reference the specific code is also listed on this page.

Exercise Part 1: Understanding Project Structure

Imagine you've just been asked to work on the Task Manager project but have no prior knowledge of its structure or technologies.

Explore the codebase: - Examine the directory structure and files without diving deep into code - Look at configuration files (package.json, requirements.txt, pom.xml) - Skim the main files to get a sense of the project

Form initial understanding: - Write down your best guess about how the codebase is organized - List the technologies and frameworks you think it uses - Identify what you think are the main components

Apply the Project Structure Prompt: - Use the "Understanding Project Structure and Technology Stack" prompt with AI - Include your initial understanding and questions - Compare the AI's analysis with your own observations

Document your findings: - Record any misconceptions you had - Note important entry points and architectural patterns identified - List the key components and their responsibilities

Exercise Part 2: Finding Feature Implementation

Imagine your team lead has asked you to add a new feature: "Task Export to CSV". You need to first understand how similar features are implemented.

Initial search: - Search for any existing export or file-related functionality in the codebase - Look for code related to data transformation or external file operations - Identify any utility functions that might be reusable for file operations

Form a hypothesis: - Based on your initial search, write down where you think task data export functionality might belong - Note which existing components might need to be modified - List search terms you used and files you found

Apply the Feature Location Prompt: - Use the "Finding Feature Implementation Locations" prompt with AI - Include your search approach and findings so far - Ask for guidance on locating similar features or patterns

Document your findings: - Map out where in the codebase you would implement this new feature - Note related components that would be affected - Outline a plan for how you would approach implementing the export feature

Exercise Part 3: Understanding Domain Model

Imagine you need to understand the Task Manager's domain model to implement new business rules.

Extract domain model: - Identify the core entity classes (Task, TaskStatus, TaskPriority, etc.) - Look for business logic related to tasks - Note any terminology or concepts that seem specific to this application

Form initial understanding: - Sketch a simple diagram of how you think the entities relate - Write a brief explanation of what you think each entity represents - Note any questions or confusion you have about the business logic

Apply the Domain Understanding Prompt: - Use the "Understanding Domain Models and Business Concepts" prompt with AI - Include your current understanding and specific questions - Ask the AI to test your understanding with questions

Test your knowledge: - Answer the questions the AI poses about the domain model - Revise your entity diagram based on new understanding - Create a glossary of domain terms used in the application

Exercise Part 4: Practical Application

Now, apply your understanding to a practical scenario:

Scenario: Your team needs to implement a new business rule: "Tasks that are overdue for more than 7 days should be automatically marked as abandoned unless they are marked as high priority."

Planning: Based on your understanding from the previous parts: - Identify which files you would need to modify - Outline the changes you would make to implement this rule - Note any questions you would ask your team before implementing

Reflection: - How did the AI prompts help you understand where and how to implement this feature? - What aspects of the codebase are you still unsure about? - What would be your next steps to deepen your understanding?

Final Discussion and Reflection

Group discussion: - Share your approach to understanding the codebase - Discuss challenges you encountered and how you overcame them - Compare different strategies used by others in the group

Personal reflection: - Which prompt was most helpful for building your understanding? - What would you do differently next time you approach an unfamiliar codebase? - What additional tools or resources would complement the AI prompting approach?

Submission

Create a short document (1-2 pages) summarizing: - Your initial vs. final understanding of the Task Manager codebase - The most valuable insights gained from each prompt - Your approach to implementing the new business rule - Any strategies you've developed for approaching unfamiliar code in the future

Submission for Exercise 1 — Knowing where to start:

Create a short document (1-2 pages) that includes:

1. Your initial vs. final understanding of the Task Manager codebase (structure, technologies, main components).
2. Your documented findings from each part — project structure misconceptions and entry points (Part 1), where the CSV export feature would live (Part 2), the domain-model diagram and glossary (Part 3).
3. Your plan for implementing the new business rule ("tasks overdue > 7 days → abandoned unless high priority"): which files you'd modify and the changes you'd make.
4. The most valuable insight gained from each of the three prompts.
5. Any strategies you've developed for approaching unfamiliar code in the future.

Suggested prompt *(for Exercise 1 — Part 1, "Understanding Project Structure and Technology Stack"):*

> I'm a junior developer who just joined this project (the Task Manager application) and I'm doing an exercise to understand an unfamiliar codebase. I've read the README but still need help understanding the project structure and technology stack.
>
> Here's my current understanding of the project: - It seems to be a [your best guess at what the application does] - It appears to use [technologies/frameworks you've identified] - The folder structure seems to follow [pattern you've observed]
>
> Project structure: [Paste concise directory structure — limit to top-level folders and key subfolders]
>
> Key configuration files: [Paste relevant sections from package.json / pom.xml / requirements.txt]
>
> Could you: (1) validate my understanding and correct any misconceptions; (2) identify additional key technologies, frameworks, and libraries used; (3) explain what each main folder likely contains and its purpose; (4) point out where the application entry points are located; (5) suggest 3-5 specific questions I should ask my team. I'm particularly confused about [specific areas]. After your explanation, suggest a small exploration exercise I could do to verify my understanding.

(For Parts 2-4, reuse the "Finding Feature Implementation Locations" prompt for the CSV export feature, the "Understanding Domain Models and Business Concepts" prompt for the domain model, and combine both to plan the "overdue > 7 days → abandoned" business rule.)

Exercise 2 — Codebase Exploration Challenge

Exercise Overview

In this exercise, you'll practice applying the three prompt strategies we've learned to understand existing code without making changes. You'll work with a task management application implemented in Python, JavaScript, or Java (choose your preferred language).

The goal is to gain a deep understanding of how the code works through systematic exploration using our AI prompt templates.

Check out the starter code at this Gitlab Repo.

Setup

Download the code for your preferred language (or fork the applicable repo): - Python: WTC Github - Python Task Manager - JavaScript: WTC Github - JavaScript Task Manager - Java: WTC Github - Java Task Manager

Review the basic structure of the code: - Don't try to understand everything yet - Just identify the main files and what they might do based on names

Create a new document called "Code Understanding Journal" where you'll record your findings.

Exercise Part 1: Understanding a Specific Feature

In this part, use the "Prompt 1: Understand how a specific feature works" prompt to understand how task creation and status updates work. - Identify the files related to task creation and status updates - Copy relevant code snippets into your prompt - Use the prompt template to ask the AI to explain this feature

Record in your journal: - The main components involved in task creation/updates - The execution flow when a task is created/updated - How data is stored and retrieved - Any interesting design patterns you discovered

Exercise Part 2: Deepen Understanding Through Guided Questions

In this part, use the "Prompt 2: Deepen understanding of a codebase" prompt to explore the task prioritization system. - Read through the code to form an initial understanding of how task priorities work - Use the prompt template to share your understanding with the AI and get guided questions - Answer those questions by examining the code further

Record in your journal: - Your initial understanding vs. what you discovered - The key insights the guided questions helped you uncover - Any misconceptions you had that were clarified

Exercise Part 3: Mapping Data Flow

In this part, use the "Prompt 3: Mapping Data Flow and State Management" prompt to understand how data flows through the application when a task is marked as complete. - Identify the entry points and components involved when marking a task as complete - Find code that handles state changes for task completion - Use the prompt template to ask the AI to help map the complete data flow

Record in your journal: - A diagram of the data flow (can be hand-drawn or digital) - The state changes that occur during task completion - Potential points of failure in this process - How the application persists these changes

Exercise Part 4: Reflection and Presentation

Review your journal and consolidate your understanding.

Prepare a 3-5 minute presentation that includes: - A high-level overview of the application architecture - How the three key features work (task creation, prioritization, and completion) - One interesting design pattern or approach you discovered - What you found most challenging to understand and how the prompts helped

Share your presentation with the group, focusing on your process for gaining understanding rather than just explaining the code.

Tips for Success - Focus on understanding, not critiquing or improving the code - Use precise, specific questions in your prompts - Include enough code context for the AI to give meaningful responses - Document your process, including false starts or misconceptions -

Draw diagrams to help visualize relationships between components - Use the AI to validate your understanding, not just to explain the code

Expected Outcomes

By the end of this exercise, you should be able to: - Explain how the task manager application works at a high level - Describe the data flow for key features - Understand how the code organizes and manages state - Demonstrate effective use of AI prompts to understand unfamiliar code - Identify which prompt strategies work best for different understanding needs

Submission for Exercise 2 — Codebase Exploration Challenge:

Produce a "Code Understanding Journal" plus a short presentation, covering:

1. Part 1 — task creation/status updates: main components involved, the execution flow, how data is stored/retrieved, and any design patterns discovered.
2. Part 2 — task prioritization: your initial understanding vs. what you discovered, and the misconceptions the guided questions clarified.
3. Part 3 — task completion: a data-flow diagram, the state changes that occur, potential points of failure, and how changes are persisted.
4. Part 4 — a 3-5 minute presentation: high-level architecture overview, how the three features work, one interesting design pattern, and what was most challenging (and how the prompts helped).

Suggested prompt *(for Exercise 2 — Part 1, "Understand how a specific feature works"):*

> I'm trying to understand how task creation and status updates work in this Task Manager [language/framework] codebase. This feature seems to handle [brief description of what you think it does].
>
> Here are the key files that appear to be involved: 1. [filepath 1] — Seems to be [your guess about purpose] 2. [filepath 2] — Might be handling [your guess]
>
> Here are code snippets from parts I'm struggling to understand: [paste snippets]
>
> Could you: (1) explain in simple terms what this component actually does; (2) walk me through the execution flow when a task is created/updated; (3) clarify how these files interact; (4) identify any external dependencies; (5) explain any complex code blocks; (6) provide a mental model I can use. I'm particularly confused about [specific aspect]. Finally, suggest 3 small code changes I could make to validate my understanding — don't give me the change, give me the requirement.

(For Parts 2 and 3, use the "Deepen understanding of a codebase" prompt on the prioritization system, then the "Mapping Data Flow and State Management" prompt on the task-completion flow.)

Exercise 3 — Algorithm Deconstruction Challenge

Exercise Instructions

Select one of the practical algorithms below (examples in Python, JavaScript, and Java): - Task priority sorting and filtering algorithm - Task text parser (converting free-form text to structured tasks) - Task list merging (two-way sync between task stores)

Use the provided AI prompts from the "Deciphering Complex Functions and Algorithms" section to break down the algorithm.

Document your understanding of the AI explanation (use visual diagrams where it helps).

Document insights and learning points.

Check out the starter code at this Gitlab Repo. For reference the specific code is also listed on this page.

Reflection Questions - How did the AI's explanation change your understanding of the algorithm? - What aspects were still difficult to understand after AI explanation? - How would you explain this algorithm to another junior developer? - Did you test this understanding against AI? - How might you improve the algorithm based on your understanding?

Provided algorithms (reference code for each is given on the course page in Python, JavaScript, and Java): - **Algorithm 1: Task Priority Sorting Algorithm** — sorts tasks based on a weighted scoring system that considers priority, due date proximity, and completion status (`calculate_task_score`, `sort_tasks_by_importance`, `get_top_priority_tasks`). - **Algorithm 2: Task Text Parser** — parses free-form text (e.g. `"Buy milk @shopping !2 #tomorrow"`) to extract task properties, where basic text is the title, `@tag` adds a tag, `!N` or `!name` sets priority, and `#date` sets a due date (`parse_task_from_text`, `get_next_weekday`). - **Algorithm 3: Task List Merging (Two-Way Sync)** — merges task lists from two sources, handling conflicts and synchronizing changes with "most recent update wins" plus special handling for completed status and tag unions (`merge_task_lists`, `resolve_task_conflict`).

Submission for Exercise 3 — Algorithm Deconstruction Challenge:

1. The algorithm you selected (priority sorting, text parser, or task-list merging) and language.
2. Your documented understanding of the AI's explanation, using visual diagrams where they help.
3. Your insights and learning points from deconstructing the algorithm.
4. Your answers to the reflection questions (how the explanation changed your understanding, what was still difficult, how you'd explain it to another junior dev, whether you tested your understanding against AI, and how you'd improve the algorithm).

Suggested prompt *(for Exercise 3 — "Understand an Algorithm Through Step-by-Step Analysis"):*

> I'm trying to understand this algorithm/function in our Task Manager codebase (the [Task Priority Sorting / Task Text Parser / Task List Merging] algorithm):
>
> [Paste the complex function here]
>
> Based on my current understanding: (1) I think this function is trying to [your best guess at purpose]; (2) the inputs seem to be [parameters] and it returns [what you think it returns]; (3) I'm particularly confused about [specific part].
>
> Could you help me understand this by: (1) breaking down the algorithm into key sections with their purposes; (2) walking through a simple example execution with concrete values; (3) explaining the core technique/pattern being used; (4) highlighting any non-obvious optimizations or tricks. After your explanation, ask me 2-3 targeted questions that test my understanding of this algorithm's underlying principles, edge cases, and performance characteristics.

Chapter: Generating and Improving Documentation with AI

Exercise 4 — Code Documentation

Overview

In this exercise, you'll practice using AI to generate comprehensive documentation for undocumented code. You'll apply prompt strategies to create function/method documentation that helps other developers understand and use the code correctly.

Exercise Options

Use the starter code used in the Code Comprehension challenge, and choose one of the more complex algorithms (calculate task priority score, text parser, merging task lists).

You can use the same starter code as for the previous Code Comprehend exercise.

Instructions - Select the code you want to document - Apply Prompt 1 to generate comprehensive documentation - Review the generated documentation for accuracy and completeness - Adjust your prompt if needed to improve the results - Try Prompt 2 to get a different perspective on the code's intent and logic - Compare and combine the best elements from both documentation versions - Share your final documentation with a partner and discuss differences

Prompt Template Reminders

Prompt 1: Comprehensive Function Documentation

> Please create comprehensive documentation for this function following [JSDoc/Python docstring/etc.] conventions: `[language] [Paste function code here]` The documentation

> should include: (1) a clear description of what the function does; (2) all parameters with types and descriptions; (3) return value with type and description; (4) any exceptions or errors that might be thrown; (5) example usage; (6) any important notes or edge cases developers should be aware of.

Prompt 2: Intent and Logic Explanation

> I need help documenting the intent and logic behind this code. Please: `[language] [Paste code here]` (1) Explain what this code is trying to accomplish at a high level; (2) break down the logic step-by-step; (3) identify any assumptions or edge cases in the implementation; (4) suggest inline comments for complex parts; (5) note any potential improvements while maintaining the original functionality.

What to Submit

At the end of the exercise, you should have: - The original code you chose to document - The documentation you generated using Prompt 1 - The insights or improvements you identified using Prompt 2 - A final combined documentation version

Be prepared to share what you learned about: - Which parts of the documentation were most challenging for the AI - What additional information you needed to provide in your prompts - How you would use this approach in your own projects

Submission for Exercise 4 — Code Documentation:

1. The original code you chose to document.
2. The documentation you generated using Prompt 1 (Comprehensive Function Documentation).
3. The insights or improvements you identified using Prompt 2 (Intent and Logic Explanation).
4. A final combined documentation version.
5. Notes on what was most challenging for the AI, what extra context you had to provide, and how you'd use this approach in your own projects.

Suggested prompt *(for Exercise 4 — using "Prompt 1: Comprehensive Function Documentation"):*

> Please create comprehensive documentation for this function from the Task Manager codebase, following [Python docstring / JSDoc / Javadoc] conventions: `[language] [Paste the calculate_task_score / parse_task_from_text / merge_task_lists function here]` Include: a clear description of what the function does; all parameters with types and descriptions; the return value with type and description; any exceptions that might be thrown; an example usage; and any important notes or edge cases developers should be aware of. Then, in a second pass, explain the high-level intent, break the logic down step-by-step, and suggest inline comments for the complex parts.

Exercise 5 — API Documentation

Overview

In this exercise, you'll practice using AI to create comprehensive API documentation that helps other developers understand and use your APIs effectively. You'll apply different prompt strategies to document endpoints, create examples, and explain authentication and error handling.

Exercise Options

You have three options for this exercise: - Document your own API: Select an endpoint from an API you're working on - Document a JavaScript/Express API example: Use the provided Express.js example (a Product API endpoint with filtering and pagination) - Document a Python/Flask API example: Use the provided Flask example (a User Registration endpoint)

Instructions - Select the API endpoint you want to document - Apply Prompt 1 to generate comprehensive endpoint documentation - Review the documentation for completeness and accuracy - Apply Prompt 2 to convert the documentation to a different format (e.g., Markdown to OpenAPI) - Apply Prompt 3 to create a developer-friendly usage guide for the endpoint - Compare your documentation with a partner and provide feedback

You do NOT have to actually run the exercise code or create a runnable project from it. The exercise is only about using extracts from the code to create the documentation.

(The course page also provides the full Express and Flask example code, plus example documentation outputs in Markdown and OpenAPI/Swagger format for reference.)

Prompt Template Reminders

Prompt 1: Endpoint Documentation Generation — Create comprehensive documentation for an API endpoint (method + path + implementation code), including: a clear description of the endpoint's purpose; request parameters (path, query, body) with types and descriptions; response format with status codes and examples; authentication requirements; potential error responses with codes and messages; at least 2 example requests with their responses; and any rate limiting or special considerations.

Prompt 2: API Reference Conversion — Convert API information into a structured [Swagger/OpenAPI/API Blueprint/Markdown] document with endpoint definitions, request parameters and body schemas, response schemas with status codes, example requests/responses, and error handling information.

Prompt 3: API Usage Guide Creation — Create a developer guide explaining how to authenticate, properly format requests, handle and interpret responses, deal with common errors, and including example code in [language], tailored to a target audience and tone.

What to Submit

At the end of the exercise, you should have: - The original API endpoint code you chose to document - The comprehensive endpoint documentation you generated using Prompt 1 -

The converted documentation format you created using Prompt 2 - The developer usage guide you created using Prompt 3

Be prepared to share what you learned about: - Which parts of the API were most challenging to document - How you adjusted your prompts to get better results - Which documentation format you found most effective for your API - How you would incorporate this approach into your development workflow

Submission for Exercise 5 — API Documentation:

1. The original API endpoint code you chose to document.
2. The comprehensive endpoint documentation you generated using Prompt 1.
3. The converted documentation format you created using Prompt 2 (e.g. Markdown → OpenAPI).
4. The developer usage guide you created using Prompt 3.
5. Notes on which parts were most challenging, how you adjusted your prompts, which format was most effective, and how you'd fit this into your workflow.

Suggested prompt *(for Exercise 5 — using "Prompt 1: Endpoint Documentation Generation"):*

> Please create comprehensive documentation for this API endpoint:
>
> Endpoint: [HTTP method] [endpoint path]
>
> Implementation code: `[language] [Paste the Express Product endpoint or Flask register_user endpoint code]`
>
> Please include: (1) a clear description of the endpoint's purpose; (2) request parameters (path, query, body) with types and descriptions; (3) response format with status codes and examples; (4) authentication requirements; (5) potential error responses with codes and messages; (6) at least 2 example requests with their responses; (7) any rate limiting or special considerations. Then convert it into an OpenAPI/Swagger document, and finally produce a friendly developer usage guide with example request code.

Exercise 6 — README and User Guide Documentation

Overview

In this exercise, you'll practice using AI to create comprehensive project documentation, including README files and user guides. You'll apply different prompt strategies to document a project's features, installation process, and usage instructions.

Exercise Options

You have several options for this exercise: - Document a JavaScript/Node.js project: Use one of the provided project examples (EventHub, SimpleCMS, or TaskCLI) - Document a Python project: Use one of the provided project examples (DataInsight, ScrapeMaster, or MLToolkit) - Document the Task Manager project: Use the Task Manager system from the Code Comprehension exercise.

Instructions - Select the project you want to document - Apply Prompt 1 to generate a comprehensive README file - Review the README for completeness and accuracy - Apply Prompt 2 to create a step-by-step guide for one of the project's features - Apply Prompt 3 to create an FAQ document for the project - Compare your documentation with a partner and provide feedback

(The course page provides full project-info sheets for each example above, plus example README, step-by-step guide, and FAQ outputs for reference.)

Prompt Template Reminders

Prompt 1: Project README Generation — Create a comprehensive README.md from project name, description, key features, technologies used, and installation requirements. It should include: clear project title and description; installation instructions; basic usage examples; features overview; configuration options; troubleshooting section; contributing guidelines; and license information, plus a code structure overview.

Prompt 2: Step-by-Step Guide Creation — Create a step-by-step guide for a specific task/ process that: starts with prerequisites or required access; breaks the process into clear numbered steps; includes screenshots or code blocks where indicated; highlights potential issues or common mistakes; and ends with a troubleshooting section — tailored to the user's experience level.

Prompt 3: FAQ Document Generation — Create a comprehensive FAQ document covering getting started, common features and functionality, troubleshooting common issues, and a specific area of interest, with a clear concise answer for each question.

What to Submit

At the end of the exercise, you should have: - The project information you chose to document - The comprehensive README you generated using Prompt 1 - The step-by-step guide you created using Prompt 2 - The FAQ document you created using Prompt 3

Be prepared to share what you learned about: - Which aspects of the project were most challenging to document - How you adjusted your prompts to get better results - What you learned about document structure and organization - How you would incorporate this approach into your development workflow

Submission for Exercise 6 — README and User Guide Documentation:

1. The project information you chose to document.
2. The comprehensive README you generated using Prompt 1.
3. The step-by-step guide you created using Prompt 2 (for one of the project's features).
4. The FAQ document you created using Prompt 3.
5. Notes on which aspects were most challenging, how you adjusted your prompts, what you learned about document structure, and how you'd use this in your workflow.

Suggested prompt *(for Exercise 6 — using "Prompt 1: Project README Generation"):*

> Please create a comprehensive README.md file for my project based on the following information:
>
> Project name: [Name] · Description: [Brief description] Key features: [Feature 1], [Feature 2], ... Technologies used: [languages/frameworks] · Installation requirements: [prerequisites] Code structure overview: [paste project structure]
>
> The README should include: (1) a clear project title and description; (2) installation instructions; (3) basic usage examples; (4) features overview; (5) configuration options; (6) a troubleshooting section; (7) contributing guidelines; (8) license information. After the README, create a step-by-step guide for one key feature and an FAQ document for the project.

Chapter: Using AI to debug errors

Exercise 7 — Error Diagnosis Challenge

Exercise Instructions - Select one of the following error scenarios (sample errors provided below) - Use the debugging prompts from the "Tracing Error Messages and Stack Traces" section to analyze the error - Document the following about solving the error (you don't actually have to create fixed runnable code for this exercise): - Error description and what it means - Root cause identification - Suggested solution - Learning points that could help prevent similar errors - Share your analysis with peers to compare different approaches

Check out the starter code at this Gitlab Repo. For reference the specific code is also listed on this page.

Reflection Questions - How did the AI's explanation compare to documentation you found online? - What aspects of the error would have been difficult to diagnose manually? - How would you modify your code to provide better error messages in the future? - Did the AI help you understand not just the fix, but the underlying concepts?

Sample Error Scenarios (full error messages and code context provided on the course page): 1. Off by One Error (Python) — `IndexError: list index out of range` from a `range(len(items) + 1)` loop in an inventory report. 2. Stack Overflow (Java) — `java.lang.StackOverflowError` from a `calculateFactorial` recursion with a missing/incorrect base case. 3. Null Pointer Reference (Java) — `NullPointerException` when a `null` product is added to a `ShoppingCart` and `getPrice()` is called. 4. Index Out of Bounds (JavaScript) — `TypeError: Cannot read properties of undefined (reading '0')` when `renderUserList` assumes 5 users but only 3 exist. 5. Out of Memory (Python) — `MemoryError` when an image processor allocates huge float64 arrays for every image without releasing memory. 6. Global Variable Being Overwritten (JavaScript) — `TypeError: Cannot read properties of undefined (reading 'map')` caused by a local `let tasks` shadowing the global `tasks` array.

Example Exercise Solution Format — Structure your response as: Error Description (plain language), Root Cause, Solution (specific code changes), Learning Points.

Submission for Exercise 7 — Error Diagnosis Challenge:

For your chosen error scenario, document (you don't have to produce fixed runnable code):

1. The error description and what it means.
2. The root cause identification.
3. The suggested solution.
4. Learning points that could help prevent similar errors in future.
5. Your answers to the reflection questions (how the AI compared to online docs, what would have been hard to diagnose manually, how you'd improve error messages, and whether the AI helped you understand the underlying concepts).

Suggested prompt *(for Exercise 7 — using "Error Message Translation" / "Root Cause Analysis"):*

> I'm getting the following error and I want to understand it, not just patch it. Please translate this error message into plain language, then work with me on the root cause.
>
> Error message / stack trace: `[Paste the full error and stack trace for your chosen scenario]` Relevant code: `[language] [Paste the code context]`
>
> Could you: (1) explain what this error actually means in plain language; (2) identify the most likely root cause and point to the exact line; (3) suggest a specific fix; (4) give me learning points and defensive-coding techniques that would prevent this class of error in future. Please also explain the underlying concept, not just the one-line fix.

Exercise 8 — Performance Optimization Challenge

Exercise Instructions - Select one of the following performance scenarios (each designed for a different performance issue): - Slow Code Analysis: Python data processing function - Memory Usage Optimization: Java image processing application - Slow Database Query Analysis: JavaScript/Node.js application with database access - Use the corresponding AI prompt from the "Identifying Performance Bottlenecks" section to analyze the performance issue - Implement the suggested optimizations - Measure performance before and after your changes - Document your optimization process, results, and key learnings

Check out the starter code at this Gitlab Repo. For reference the specific code is also listed on this page.

Sample Performance Issues for Analysis (full code and context provided on the course page): 1. **Slow Code Analysis (Python)** — a `find_product_combinations` function using nested O(n²) loops over 5,000+ products, taking ~20-30 seconds on the "Product Recommendations" page (Python 3.9, 4GB RAM). 2. **Memory Usage Optimization (Java)** — an `ImageProcessor` that loads all images into memory before processing, throwing `OutOfMemoryError: Java heap space` on 50-100 high-resolution images with default 256MB JVM settings (Java 11, 8GB RAM). 3. **Slow Database Query Analysis (JavaScript/ Node.js)** — a `getCustomerOrderDetails` PostgreSQL query with correlated sub-queries and

no extra indexes, taking 8-10 seconds and causing timeouts (orders ~100k rows, order_items ~500k rows; Node.js 14, 4 cores, 8GB RAM).

Reflection Questions - How did the optimization change your understanding of the algorithm, memory management, or database query patterns? - What performance improvements did you achieve? Were they significant enough to justify the code changes? - What did you learn about performance bottlenecks that you didn't know before? - How would you approach similar performance issues in the future? - What tools or techniques would you use to identify similar issues proactively?

Submission for Exercise 8 — Performance Optimization Challenge:

1. The performance scenario you selected (slow Python code, Java memory usage, or slow DB query).
2. Your analysis of the performance issue using the corresponding AI prompt.
3. The optimizations you implemented.
4. Performance measurements before and after your changes.
5. Documentation of your optimization process, results, and key learnings (plus answers to the reflection questions).

Suggested prompt *(for Exercise 8 — using "Slow Code Analysis" / "Memory Usage Optimization" / "Slow Database Query Analysis"):*

> I have a piece of code that's running slowly and I'd like to understand why and how to improve it.
>
> Here's the slow-performing code: `[language] [Paste your chosen scenario's code]` Context about the issue: what the code is supposed to do is [purpose]; typical input size is [data]; current performance is [time/memory]; environment is [runtime + hardware].
>
> Could you please: (1) explain in simple terms why this code might be slow (or memory-hungry); (2) identify the specific operations or patterns causing the slowdown; (3) suggest 2-3 specific improvements; (4) explain the performance concepts I should learn to avoid similar issues; (5) suggest tools or techniques to measure the actual bottlenecks. I'm interested in learning the underlying performance concepts, not just a quick fix.

Exercise 9 — AI Solution Verification Challenge

Exercise Instructions - Select one of the following scenarios: - A sorting function with a subtle bug - Ask an AI tool to solve the problem - Apply all three verification strategies: - Collaborative Solution Verification - Learning Through Alternative Approaches - Developing a Critical Eye - Document the verification process and what you learned - Implement the final, verified solution

Check out the starter code at this Gitlab Repo. For reference the specific code is also listed on this page.

Sample Problem for Verification — A buggy `mergeSort` / `merge` function in JavaScript where, in the "copy remaining elements" loop, `j` is incremented instead of `i`, so the leftover left-array elements are never correctly appended.

Reflection Questions - How did your confidence in the solution change after verification? - What aspects of the AI solution required the most scrutiny? - Which verification technique was most valuable for your specific problem?

Submission for Exercise 9 — AI Solution Verification Challenge:

1. The scenario you chose and the AI-generated solution you asked it to produce.
2. Your documented verification process applying all three strategies (Collaborative Solution Verification, Learning Through Alternative Approaches, Developing a Critical Eye).
3. What you learned during verification.
4. The final, verified solution you implemented.
5. Your answers to the reflection questions (how your confidence changed, what needed the most scrutiny, which technique was most valuable).

Suggested prompt *(for Exercise 9 — using "Collaborative Solution Verification" and "Developing a Critical Eye"):*

> I asked an AI to fix this sorting function, and I want to rigorously verify the solution rather than trust it blindly. Here is the code (it has a subtle bug): `javascript [Paste the mergeSort / merge function]`
>
> Please help me verify it: (1) trace through the merge step with a concrete example (e.g. left = [1, 4], right = [2, 3]) and show what actually gets produced; (2) identify the exact bug and why it breaks correctness; (3) propose a corrected version and an alternative approach so I can compare; (4) suggest test cases (including edge cases like empty arrays, duplicates, and already-sorted input) that would catch this bug. Don't just assert it's fixed — show me the reasoning and how I could prove correctness myself.

Chapter: Using AI to help with testing

Exercise 10 — Using AI to help with testing

This exercise will guide you through the process of using AI to help improve your testing skills using the task prioritization algorithm provided in the course materials. You'll apply the strategies discussed in the chapter to develop better testing practices.

Choose one implementation of the task prioritization algorithm (See Python, JavaScript, or Java Task Manager in references GitHub repo) to work with throughout this exercise.

Check out the starter code at this Gitlab Repo.

Part 1: Understanding What to Test

Exercise 1.1: Behavior Analysis — Examine the `calculateTaskScore` function from your chosen language implementation. Use the following prompt with an AI assistant to help analyze what to test:

> I'm learning how to test this function, and I want to understand what behaviors I should test: [Paste the calculateTaskScore function] Rather than generating tests for me, please: (1) ask me questions about what I think this function does; (2) after I answer, help identify any behaviors I missed; (3) ask me what edge cases I think should be tested; (4) help me identify additional edge cases I didn't think of; (5) ask me which test I should write first and why.

Engage in the conversation with the AI, answering its questions about the function's behavior. Based on the conversation, create a list of at least 5 test cases you should write, including any edge cases identified.

Exercise 1.2: Test Planning — Now focus on the entire set of functions: `calculateTaskScore`, `sortTasksByImportance`, and `getTopPriorityTasks`. Use this prompt to help plan your testing approach:

> I'm learning to write tests for these related functions: [Paste all three functions from your chosen implementation] Instead of writing tests for me, please: (1) help me create a testing plan by asking me questions; (2) for each behavior I identify, ask me how I would test it; (3) if I miss something important, give me hints rather than answers; (4) for each edge case we identify, ask me what I expect to happen; (5) help me create a checklist of tests I should write, organized by priority.

Create a structured test plan document based on your conversation, including: priority of test cases; types of tests needed (unit, integration); test dependencies; and expected outcomes for each test.

Part 2: Improving a Single Test

Exercise 2.1: Writing Your First Test — Write a basic test for the `calculateTaskScore` function. Intentionally make it simple (just test the basic functionality). Use this prompt to improve your test:

> I wrote this test for the following function: Function: [Paste calculateTaskScore function] My test: [Paste your simple test] Instead of rewriting it for me, please: (1) ask me questions about what my test is trying to verify; (2) help me identify if my test is checking behavior or implementation details; (3) suggest how I could make the test's purpose clearer; (4) ask me what edge cases my test might be missing; (5) guide me in improving my assertions to be more precise.

Based on the feedback, rewrite your test to make it more robust and clearer.

Exercise 2.2: Learning From Examples — Choose another aspect of the `calculateTaskScore` function to test (e.g., due date calculations). Use this prompt:

> I'm trying to understand how to better test the due date calculation portion of this function: [Paste calculateTaskScore function] I'm thinking of writing a test like this (pseudocode): [Write a rough outline/pseudocode of your test idea] Please: (1) explain the principles of a good test for this specific functionality; (2) show me ONE example of a better test with comments explaining why it's better; (3) ask me questions about how I would improve my approach based on this example; (4) challenge me to identify what edge cases I should add; (5) guide me in writing more precise assertions.

Write a comprehensive test for the due date calculation functionality based on your conversation.

Part 3: Test-Driven Development Practice

Exercise 3.1: TDD for a New Feature — Imagine you need to add a new feature to the task priority system: tasks assigned to the current user should get a score boost of +12. Use this prompt before writing any code:

> I want to practice Test-Driven Development to add a new feature: Tasks assigned to the current user should get a score boost of +12. Here's the current function: [Paste calculateTaskScore function] Instead of writing code and tests for me, please: (1) ask me what I think the first test should be and why; (2) give me feedback on my proposed test; (3) after I write the test, ask me what minimal code would make it pass; (4) once I've implemented the code, guide me on what test to add next; (5) help me understand when it's time to refactor vs. add new functionality.

Follow the TDD process: write a failing test for the new feature; implement the minimal code to make it pass; refactor if necessary; write the next test if needed.

Exercise 3.2: TDD for Bug Fix — Imagine there's a bug in the existing code: the calculation for "days since update" is incorrect and should be using `.days` instead of dividing by milliseconds (in JavaScript) or using a different ChronoUnit (in Java). Use this prompt:

> I want to practice TDD to fix a bug: The calculation for "days since update" isn't working correctly. Here's the current function: [Paste calculateTaskScore function] Please: (1) ask me what test I would write to reproduce the bug; (2) help me understand if my test actually demonstrates the bug; (3) guide me in implementing a minimal fix; (4) ask me if there are any other tests I should add to prevent regression.

Write a test that demonstrates the bug, then fix the code to make the test pass.

Part 4: Integration Testing

Exercise 4.1: Testing the Full Workflow — Think about how to test the three functions working together: `calculateTaskScore`, `sortTasksByImportance`, and `getTopPriorityTasks`. Use this prompt:

> I want to create an integration test for the task priority workflow: [Paste all three functions] Rather than writing the test for me, please: (1) ask me what scenarios an integration test should verify; (2) guide me in designing test data that would exercise the entire workflow; (3) help me understand what assertions would verify the correct behavior; (4) ask me how I would structure the test to make it readable and maintainable.

Write an integration test that verifies the three functions work correctly together.

Discussion

After completing all parts of this exercise, prepare a short document that includes: your test plan document from Part 1; your improved unit tests from Part 2; your TDD implementation and tests from Part 3; your integration test from Part 4; and a reflection on what you learned about testing through this exercise. Submit this to the facilitator, and discuss this in the group discussion.

Submission for Exercise 10 — Using AI to help with testing:

Prepare a short document that includes:

1. Your test plan document from Part 1 (priorities, test types, dependencies, expected outcomes) plus your list of at least 5 test cases.
2. Your improved unit tests from Part 2 (first test rewrite, and the due-date calculation test).
3. Your TDD implementation and tests from Part 3 (the "+12 boost for current user" feature and the "days since update" bug fix).
4. Your integration test from Part 4 verifying the three functions work together.
5. A reflection on what you learned about testing through this exercise.

Suggested prompt *(for Exercise 10 — starting point, "Behavior Analysis"):*

> I'm learning how to test the `calculateTaskScore` function from the Task Manager and I want to understand what behaviors I should test — not have tests written for me. [Paste the calculateTaskScore function] Rather than generating tests, please: (1) ask me questions about what I think this function does; (2) help identify any behaviors I missed; (3) ask what edge cases I'd test (e.g. no due date, overdue tasks, DONE/REVIEW status, priority-boosting tags, recently updated tasks); (4) help me identify additional edge cases; (5) ask which test I should write first and why. Then act as a TDD coach as I add the "+12 boost for tasks assigned to the current user" feature and fix the "days since update" bug.

Module 2 (cont.) — Using AI to refactor and enhance code

Chapter: Using AI to refactor and enhance code

Exercise 11 — Understanding What to Change with AI

This exercise will help you practice using AI to analyze and improve code. You'll work with three different code examples in Java, Python, and JavaScript. For each example, you'll apply one of the prompting strategies we've discussed to make the code better.

Setup - Open your preferred AI assistant (like Claude, ChatGPT, or GitHub Copilot) - Copy the prompt template and code example for each exercise - Follow the instructions to analyze and improve the code - Compare your AI-assisted solution with the sample solution provided - Reflect on what you learned and how you might apply it to your own code

You do NOT have to have runnable code or tests for this exercise - rather compare solutions and document your learnings and insights to share with the group.

Exercise 1: Code Readability Improvement (Java) — Work with a `UserMgr` / `U` class that uses cryptic names (`u_list`, method `a(...)`, method `f(...)`) and an unsafe SQL string concatenation. - Instructions: Use the "Code Readability Improvement" prompt template; specify that this is Java code with standard Java naming conventions; analyze the AI's suggestions for improving variable names, method names, and code structure; note which readability issues the AI identified that you might have missed.

Exercise 2: Function Refactoring (Python) — Work with a long `process_orders(orders, inventory, customer_data)` function that validates orders, applies discounts, updates inventory, computes shipping/tax, and tracks revenue all in one loop. - Instructions: Use the "Function Refactoring" prompt template; specify that this function should process orders, update inventory, and track revenue; identify how the function can be broken down into smaller, more focused functions; compare your AI-generated refactoring with your own ideas.

Exercise 3: Code Duplication Detection (JavaScript) — Work with a `calculateUserStatistics(userData)` function that repeats near-identical loops to compute averages and highest values for age, income, and score. - Instructions: Use the "Code Duplication Detection" prompt template; identify the repeated patterns in the code; see how the AI suggests consolidating these patterns; evaluate which approach might be most readable for a team of junior developers.

(Full code for all three examples is provided on the course page.)

Reflection Questions - Which prompting strategy did you find most useful? Why? - What kinds of improvements did the AI suggest that you might not have thought of? - Were there any suggestions the AI made that you disagreed with? Why? - How might you adapt these prompts for your specific codebase or tech stack? - What safeguards would you put in place before applying AI-suggested refactoring to production code?

Next Steps — Try these prompts on code from your own projects; create a personal template library with these and other useful prompts; practice explaining the reasoning behind refactoring decisions; remember to always have tests in place before refactoring real code.

Submission for Exercise 11 — Understanding What to Change with AI:

You don't need runnable code or tests — compare solutions and document your learnings and insights to share with the group:

1. Example 1 (Java readability): the AI's renaming/structure improvements and which issues you'd missed.
2. Example 2 (Python function refactoring): how you'd break `process_orders` into smaller functions, compared with your own ideas.
3. Example 3 (JavaScript duplication): the repeated patterns and how the AI consolidated them.
4. Your answers to the reflection questions (most useful strategy, unexpected suggestions, anything you disagreed with, how you'd adapt the prompts, and what safeguards you'd apply before refactoring production code).

Suggested prompt *(for Exercise 11 — Exercise 1, "Code Readability Improvement"):*

> This is Java code that follows (or should follow) standard Java naming conventions. Please help me improve its readability without changing its behavior: `java [Paste the UserMgr / U classes]` Could you: (1) suggest clearer names for the classes, fields, and methods (e.g. `UserMgr` → `UserManager`, `a(...)`, `f(...)`); (2) identify structural and readability issues, including the unsafe SQL string concatenation; (3) explain each change and why it improves readability; (4) point out any issues I might have missed. For Exercises 2 and 3, use the "Function Refactoring" prompt to decompose `process_orders`, and the "Code Duplication Detection" prompt to consolidate the repeated loops in `calculateUserStatistics`.

Exercise 12 — Function Decomposition Challenge

Exercise Instructions - Select a complex function from your codebase or one of the provided sample functions: - Data Validation Function with Nested Conditionals (JavaScript) - Report Generation Function with Multiple Data Transformations (Python) - Data Processing Pipeline with Error Handling (Java) - Use the provided AI prompts to: - Identify the distinct responsibilities in the function - Create a decomposition plan - Extract helper functions with clear names and purposes - Refactor the main function to call the helper functions - Run tests to verify the refactoring preserves behavior - Document your refactoring approach and the benefits gained

Check out the starter code at this Gitlab Repo.

Sample Complex Functions (full code provided on the course page): 1. **`validateUserData(userData, options)` (JavaScript)** — a large validation function with deeply nested conditionals covering required fields, username, password, email, date of birth, address (with US/CA/UK postal-code formats), phone, and custom validations. 2. **`generate_sales_report(...)` (Python)** — a report generator that validates parameters, filters by date and other criteria, computes metrics, groups data, and branches into summary/detailed/forecast reports plus optional charts and multiple output formats. 3. **`processCustomerData(rawData, source, options)` (Java)** — an ETL pipeline that validates,

transforms, deduplicates, and loads customer records while tracking per-type error counts and building a processing report.

Reflection Questions - How did breaking down the function improve its readability and maintainability? - What was the most challenging part of decomposing the function? - Which extracted function would be most reusable in other contexts?

Submission for Exercise 12 — Function Decomposition Challenge:

1. The distinct responsibilities you identified in the chosen function.
2. Your decomposition plan (the helper functions and their purposes).
3. The extracted helper functions with clear names.
4. The refactored main function that calls the helpers, plus tests run to verify behaviour is preserved.
5. Documentation of your refactoring approach and the benefits gained (plus answers to the reflection questions).

Suggested prompt *(for Exercise 12 — "Function Responsibility Analysis" / "Single-Responsibility Extraction"):*

> I have a large function that does too many things, and I want to decompose it into smaller, single-responsibility functions — without changing its behavior. Here it is: `[language] [Paste validateUserData / generate_sales_report / processCustomerData]` Please: (1) list the distinct responsibilities this function currently handles; (2) propose a decomposition plan naming each helper function and what it should do; (3) show how the main function would read after calling those helpers; (4) flag any edge cases or shared state I need to preserve; (5) suggest tests that would confirm the refactor preserves the original behavior. Walk me through it step by step rather than dumping a full rewrite.

Exercise 13 — Code Readability Challenge

Exercise Instructions

Choose one (or more, if you have time) of the exercises in Sample Code, and follow these steps: - Understand the original code: Run the unit tests to make sure you understand what the code is doing - Apply the appropriate prompt: Use the AI prompt that best fits the readability issue - Implement the improvements: Refactor the code based on the AI's suggestions - Verify functionality: Run the unit tests again to ensure your refactored code still works correctly

Running unittests should be familiar to you, at least for the languages you have used. We do give some high-level pointers in Running the Unit Tests on running the unit-tests for these examples.

Reflection Questions

After receiving the AI's suggestions, think about: - How much easier is the code to understand now? - What readability issues did you miss that the AI caught? - What readability issues did the AI miss that you noticed? - Which readability improvements had

the biggest impact? - How did the improved names change your understanding of the code's purpose? - What readability patterns can you apply to your future code? - Practice explaining the readability improvements to a hypothetical team member (use the other attendees)

Sample Code (four examples, each with unit tests, provided on the course page — choose at least one; keep the unit tests passing after any change): 1. **Cryptic Variable Names (JavaScript)** — an inventory function `p(i, a, q)` with single-letter names; your task: use the "Code Readability Improvement" prompt to identify and rename the cryptic variables, work out what the function is actually doing, and improve the names to make the purpose clear. 2. **Missing Documentation (Python)** — a `calculate(principal, rate, time, additional, frequency)` compound-interest function with no docstring; your task: add proper documentation clarifying what the function does, what each parameter means (including units), and what the return values represent, and consider renaming variables for clarity. 3. **Complex Algorithm Without Comments (Java)** — a `sortItems(int[] array)` selection sort with no comments; your task: add explanatory comments covering what algorithm this is, how it works conceptually, what each section does, and its time complexity, plus any name/structure improvements. 4. **Poor Formatting and Structure (Python)** — a `discount(cart, promos, user)` function with terrible formatting; your task: improve indentation, line breaks, consistent spacing, and logical grouping, and consider breaking it into smaller functions.

Running the Unit Tests — JavaScript (Example 1): run in a browser console or Node.js, uncommenting `runTests()`. Python (Examples 2 and 4): save code and tests in separate files, import the function, and run `python -m unittest`. Java (Example 3): add JUnit to your project and run via your IDE or command line.

Submission for Exercise 13 — Code Readability Challenge:

For at least one chosen example (keeping the unit tests passing):

1. The original code and your refactored, more-readable version.
2. Confirmation the unit tests still pass after your changes.
3. Readability issues the AI caught that you had missed.
4. Readability issues you noticed that the AI missed.
5. Your reflection on which improvements had the biggest impact and what readability patterns you can apply to future code.

Suggested prompt *(for Exercise 13 — Example 1, "Code Readability Improvement"):*

> This is JavaScript code with cryptic single-letter names, and I need to keep its behavior identical (its unit tests must still pass). Please help me make it readable: `javascript [Paste the p(i, a, q) inventory function]` Could you: (1) work out what this function actually does and describe its purpose in one sentence; (2) suggest clear, descriptive names for every variable and for the function itself, and for the returned object keys (`s` , `t`); (3) show the refactored version with the improved names; (4) point out any readability issues I might have missed. Keep the logic unchanged so the existing tests still pass. (For Examples 2-4, use a documentation prompt for `calculate` , a "explain and

> comment this algorithm" prompt for `sortItems`, and a "reformat and restructure" prompt for `discount`.)

Exercise 14 — Design Pattern Implementation Challenge

Exercise Instructions - Select a piece of code (see Sample Pattern Opportunities) that exhibits one of these pattern opportunities: - Conditional logic that could use the Strategy pattern - Object creation complexity that could use the Factory pattern - Complex object relationships that could use the Observer pattern - Incompatible interfaces that could use the Adapter pattern - Use the provided AI prompts to: - Analyze the code for pattern opportunities - Get implementation guidance for the chosen pattern - Refactor the code to implement the pattern - Write tests to verify the refactored code preserves the original behavior - Document the benefits gained from the pattern implementation

Sample Pattern Opportunities - Strategy Pattern Opportunity: Complex pricing rules with many conditionals - Factory Pattern Opportunity: Complex object creation with many dependencies - Observer Pattern Opportunity: A system where changes to one object require updates to many others - Adapter Pattern Opportunity: Code that needs to work with an external API with an incompatible interface

Sample Code for Exercise (four full examples provided on the course page): 1. **Strategy Pattern Opportunity (JavaScript)** — a `calculateShippingCost(packageDetails, destinationCountry, shippingMethod)` calculator with nested conditionals for standard/express/overnight shipping and per-country rates. 2. **Factory Pattern Opportunity (Python)** — a `DatabaseConnection` class whose `connect()` method branches on `db_type` (mysql, postgresql, mongodb, redis) with complex per-type initialization. 3. **Observer Pattern Opportunity (Java)** — a `WeatherStation` whose `setMeasurements(...)` must manually update several displays (current conditions, statistics, forecast, heat index) whenever data changes. 4. **Adapter Pattern Opportunity (TypeScript)** — a `PaymentService` that talks directly to two incompatible gateway APIs (`StripeAPI` and `PayPalAPI`) behind a common `PaymentProcessor` interface.

These examples are intentionally simplified to focus on the pattern opportunities while remaining accessible to junior developers.

Reflection Questions - How did implementing the pattern improve the code's maintainability? - What future changes will be easier because of this pattern? - Were there any unexpected challenges in implementing the pattern?

Submission for Exercise 14 — Design Pattern Implementation Challenge:

1. The code you selected and the pattern opportunity you identified (Strategy, Factory, Observer, or Adapter).
2. Your analysis of the code for the pattern opportunity, plus the implementation guidance you obtained.
3. The refactored code implementing the pattern.
4. Tests written to verify the refactored code preserves the original behaviour.
5. Documentation of the benefits gained (plus answers to the reflection questions).

Suggested prompt *(for Exercise 14 — pattern analysis and implementation):*

> I have code that I think could benefit from a design pattern (I'm considering the [Strategy / Factory / Observer / Adapter] pattern). Here it is: `[language] [Paste calculateShippingCost / DatabaseConnection / WeatherStation / PaymentService]` Please: (1) confirm whether this is a good fit for that pattern (or suggest a better one) and explain why; (2) give me step-by-step implementation guidance for applying the pattern to this code; (3) show the refactored structure (interfaces/classes and how they collaborate); (4) explain what future changes become easier; (5) suggest tests that verify the refactor preserves the original behavior. Keep it at a level appropriate for a junior developer and explain the reasoning, don't just hand me the final code.

Module 3 — Learning with AI

Chapter: Deepening Knowledge of Your Current Programming Language

---

Exercise 15 — Applying AI to deepen programming language understanding

Do the activities below, and journal (i.e. document) about the three key learnings for each — we will discuss that in the group after the exercise.

Activity 1: Idiomatic Code Transformation

Objective: Learn how to write more language-idiomatic code - Select a function you've written recently (15-30 lines of code) - Use this prompt:

> I'm learning to write more idiomatic [language]. Here's my current code: [paste your code] Could you: (1) suggest ways to make this more idiomatic; (2) explain why these changes follow language best practices; (3) point out any language features I'm not taking advantage of; (4) show me both my version and an improved version side by side.

- Implement the suggestions in your codebase
- Document 3 key learnings from this exercise

Activity 2: Code Quality Detective

Objective: Develop an eye for code quality issues in your language - Find a piece of code you wrote 3+ months ago (or any older code) - Use this prompt:

> I'm a junior developer working with [language]. Could you review this code for quality improvements: [paste code] Please: (1) identify any code smells or quality issues; (2)

> suggest specific improvements; (3) explain why these improvements matter in [language]; (4) rate the code's readability, performance, and maintainability.

- Create a checklist of the issues identified
- Use this checklist in your future code reviews
- Document 3 key learnings from this exercise

Activity 3: Understanding Language Feature

Objective: Discover and learn advanced language features - Choose an area of your language you're less familiar with (e.g., Python decorators, JavaScript generators) - Use this prompt:

> I want to improve my understanding of [specific feature] in [language]. (1) Could you explain this feature with simple examples? (2) Show me 3 practical use cases where this would be valuable. (3) Provide a small project idea that would help me practice this feature. (4) What common mistakes should I avoid when using this feature?

- Implement a small code sample using this feature
- Share your implementation with AI for feedback
- Document 3 key learnings from this exercise

Group Discussion — Share your 3 key learnings for each activity, and highlight interesting code changes you made (keeping in mind the security and confidentiality policy of any code you worked on). Are there any common themes that popped out in the learnings?

Extension Activities — *Language-Specific Style Guide:* Create a personalized style guide for your language by asking AI to compile best practices, then validate it against official style guides used by your team.

Submission for Exercise 15 — Applying AI to deepen programming language understanding:

Journal (document) three key learnings for each activity:

1. Activity 1 (Idiomatic Code Transformation): the idiomatic changes made and 3 key learnings.
2. Activity 2 (Code Quality Detective): the code smells/quality issues found, a reusable checklist, and 3 key learnings.
3. Activity 3 (Understanding Language Feature): your small code sample using the feature, AI feedback on it, and 3 key learnings.
4. Notes for the group discussion on any common themes across the learnings.

Suggested prompt *(for Exercise 15 — Activity 1, "Idiomatic Code Transformation"):*

> I'm learning to write more idiomatic [your language, e.g. Python/JavaScript/Java]. Here's a 15-30 line function I wrote recently: `[language] [paste your code]` Could you: (1) suggest

> ways to make this more idiomatic; (2) explain why each change follows the language's best practices; (3) point out language features I'm not taking advantage of; (4) show my version and an improved version side by side. Then help me with the other two activities: review an older piece of my code for code smells and rate its readability/performance/ maintainability, and explain an advanced feature I'm less familiar with (e.g. decorators/ generators) with examples, use cases, a practice project idea, and common mistakes to avoid.

Chapter: Learning a New Programming Language with AI

Exercise 16 — Learning a New Programming Language with AI

Overview

This exercise will guide you through applying the structured prompting techniques described in the workshop to learn a new programming language. You'll practice creating effective prompts, verifying your understanding, and building a small project in your target language.

Prerequisites - A language you're already comfortable with (your "source language") - A target language you want to learn - Access to an AI assistant (like Claude, ChatGPT, etc.) - Development environment set up for your target language

Exercise Structure — This exercise is broken into parts that align with the prompting strategies from the workshop: create your learning journey plan; apply the four-step prompting strategy; use advanced prompting techniques; build a simple project in your new language.

Part 1: Create Your Learning Journey Plan - Define your goals: Write 2-3 specific learning goals for your target language (e.g. "Build a REST API in Go" or "Create data visualizations in Julia"). - Create a learning plan: Use AI to help create a structured learning plan like the Java example in the workshop:

> I'm proficient in [YOUR SOURCE LANGUAGE] and want to learn [TARGET LANGUAGE] to [YOUR GOAL]. Could you help me create a structured learning journey plan with: 3-4 distinct learning phases; prerequisites for each phase; 4-5 specific learning steps per phase; verification activities for each phase. Please structure it similar to this Java learning plan example: [PASTE THE JAVA LEARNING PLAN FROM THE WORKSHOP].

- Refine your plan: Review the AI-generated plan and modify it to better match your specific needs and interests.

Part 2: Apply the Four-Step Prompting Strategy (choose a specific topic from your learning plan) - *Step 1: Conceptual Understanding* — ask for philosophical differences between source and target language, problems the target language was designed to solve,

mental models to adjust, and common misconceptions. - *Step 2: Step-by-Step Breakdown* — ask for a breakdown of a specific feature/concept: how it's implemented in the target language, how it compares to a similar concept in the source language, key syntax and structures, and common patterns/best practices (no complex code yet). - *Step 3: Guided Implementation* — ask the AI to guide you through implementing a simple example, explaining each part of the syntax, especially where it differs from the source language. - *Step 4: Understanding Verification* — write the code yourself, then submit it and ask the AI to verify best practices, explain improvements, suggest what to learn next, and point out any source-language habits showing in your code.

Part 3: Practice Advanced Prompting Techniques (choose at least two): Using Context Effectively; Promoting Deep Understanding; Learning Through Teaching; Using a Tutor (where the AI asks guiding questions and gives hints rather than direct answers).

Part 4: Build a Mini-Project — Define a simple project using the concepts you've learned (a command-line text tool, a simple API endpoint, a data processing script, or a basic UI component). Ask the AI to help break the project into components, suggest libraries/frameworks, outline key files/classes, and identify challenges coming from your source language. Implement it step by step, then submit the finished project for a review of idioms, refactoring opportunities, remaining source-language patterns, and next steps.

Reflection Questions - Which prompting strategies were most effective for your learning style? - What surprised you about the target language that wasn't immediately obvious? - How did your mental models from your source language help or hinder learning? - What would you do differently in your next learning session? - What gaps remain in your understanding of the target language?

Extension Activities — Create a "cheat sheet" comparing key concepts between source and target languages; expand your mini-project; practice explaining a complex concept in your own words; analyze a small portion of an open-source project in your target language; create your own custom prompting template for learning future languages.

Submission for Exercise 16 — Learning a New Programming Language with AI:

1. Your 2-3 learning goals and your refined learning journey plan (Part 1).
2. The outputs of the four-step strategy (Part 2): conceptual understanding notes, concept breakdown, your guided implementation, and your verified code.
3. Evidence of practising at least two advanced prompting techniques (Part 3).
4. Your mini-project in the target language, plus the AI review/refactor feedback (Part 4).
5. Your answers to the reflection questions (most effective strategies, surprises, how source-language mental models helped/hindered, and remaining gaps).

Suggested prompt *(for Exercise 16 — Part 2, Step 1 "Conceptual Understanding"):*

> I'm currently proficient in [SOURCE LANGUAGE] and want to learn [TARGET LANGUAGE] to [your goal]. Before diving into code: (1) What are the key philosophical differences between [SOURCE] and [TARGET]? (2) What problems was [TARGET] designed to solve? (3) What mental models should I adjust coming from [SOURCE]? (4) What are common

> misconceptions [SOURCE] developers have about [TARGET]? After this, act as my tutor for the four-step strategy — break down a specific concept, guide me through a simple implementation, then verify the code I write and flag any [SOURCE LANGUAGE] habits, before helping me scope and review a small mini-project.

Chapter: Learn a new framework, library or API (FastAPI)

Exercise 17 — Getting Started with FastAPI

In this exercise, you'll use AI to quickly get up to speed with FastAPI, a modern Python web framework for building APIs. You'll go from zero knowledge to creating a working API with proper structure.

Part 1: Understanding FastAPI Fundamentals

Task: Use AI to learn the basics of FastAPI. Prompts to try: - "What is FastAPI and how does it compare to Flask and Django?" - "Explain the core concepts and terminology used in FastAPI." - "What are the key advantages of FastAPI over other Python web frameworks?" - "Create a glossary of essential FastAPI terms and concepts."

Goal: Take notes on FastAPI's purpose, design philosophy, and key features.

Part 2: Creating Your First API

Task: Use AI to help you build a simple "Hello World" API with FastAPI. Prompts to try: - "Generate a basic 'Hello World' example for FastAPI with comments explaining each part." - "How do I structure a FastAPI project following best practices?" - "Show me how to run and test a FastAPI application locally."

Implementation: A reference `main.py` is provided on the course page (a FastAPI app with a root endpoint, a path-parameter `/items/{item_id}` endpoint, and a query-parameter `/search/` endpoint). Save it as `main.py`, run `pip install fastapi uvicorn` then `uvicorn main:app --reload`, and visit `http://127.0.0.1:8000/docs` to see the automatic API documentation.

Part 3: Enhancing Your API

Task: Use AI to help you add more advanced features to your basic API. Prompts to try: - "How do I add request body validation with Pydantic models in FastAPI?" - "Show me how to implement proper error handling in FastAPI." - "How can I organize my FastAPI project into multiple files and modules?"

Don't know what pydantic is? Use the prompts from this chapter and from "Learning a New Programming Language with AI" to learn about Python and Pydantic. A reference multi-file project structure (models, routes, utils/exceptions) is provided on the course page.

Part 4: Exercise Challenge

Task: Using what you've learned and with AI assistance, create a simple FastAPI application for managing a to-do list with the following features: - Create a new to-do item with title, description, and due date - List all to-do items with optional filtering by status (completed/pending) - Mark a to-do item as completed - Delete a to-do item

Suggested implementation steps: define Pydantic models for the to-do items; create endpoints for CRUD operations; implement proper validation and error handling; add filtering capabilities for the list endpoint; document your API using FastAPI's automatic documentation features.

Tips for using AI effectively: start by asking for the basic structure, then iterate with more specific questions; when you encounter an error, share it with AI and ask for troubleshooting help; ask AI to explain any unfamiliar concepts you encounter; request examples of specific FastAPI features as you need them.

Submission for Exercise 17 — Getting Started with FastAPI:

1. Your notes on FastAPI's purpose, design philosophy, and key features (Part 1).
2. Your first "Hello World" API running locally, viewed via the automatic `/docs` (Part 2).
3. Your enhanced API with Pydantic request-body validation, error handling, and multi-file structure (Part 3).
4. The completed to-do list application (Part 4): create item (title, description, due date), list with status filtering, mark complete, delete — with validation, error handling, and automatic docs.

Suggested prompt *(for Exercise 17 — Part 4 challenge):*

> I'm learning FastAPI and I want to build a simple to-do list API, using you as a learning assistant rather than just a code generator. The API needs to: create a new to-do item with title, description, and due date; list all to-do items with optional filtering by status (completed/pending); mark a to-do item as completed; and delete a to-do item.
>
> Please help me step by step: (1) design the Pydantic models for a to-do item (including validation like a non-empty title and a valid due date); (2) outline the CRUD endpoints and project structure; (3) show me how to add proper validation and error handling (e.g. 404 when an item isn't found); (4) add filtering to the list endpoint. Explain each unfamiliar concept as we go, and when I paste an error, help me troubleshoot it.

Exercise 18 — Contextual Learning with FastAPI

In this exercise, you'll practice using AI to learn FastAPI through the lens of your existing Python web framework knowledge.

Part 1: Framework Comparison

Task: Use AI to compare FastAPI with frameworks you're already familiar with. Prompts to try: - "Compare FastAPI and Flask: what concepts are similar and what's fundamentally different?" - "How does FastAPI's dependency injection system compare to Django's

middleware?" - "If I'm used to Flask's Blueprint system, what's the equivalent in FastAPI?" - "How is FastAPI's request validation different from typical form validation in Django?"

Goal: Create a personal "translation table" between concepts in frameworks you know and their FastAPI equivalents.

Part 2: Understanding Design Choices

Task: Explore why FastAPI was designed the way it was. Prompts to try: - "Why did FastAPI choose Pydantic for data validation instead of creating its own validation system?" - "What problem does FastAPI's automatic API documentation solve that was challenging in other frameworks?" - "Why does FastAPI use type hints so extensively? What advantages does this provide?" - "What motivated the async-first approach in FastAPI compared to the synchronous approach of Flask?"

Goal: Write a short summary of FastAPI's design philosophy and how it influences the way you should structure applications.

Part 3: Applied Contextual Learning

Task: Use your prior knowledge and AI guidance to implement a feature in FastAPI based on your understanding of similar features in other frameworks.

Implementation Challenge: Create a simple FastAPI application that implements authentication using JWT tokens. If you've implemented authentication in other frameworks, ask AI to explain how the approach differs in FastAPI. (A reference JWT-auth `main.py` using OAuth2, Pydantic models, password hashing and dependency-injected `get_current_user` is provided on the course page.)

Reflection Questions: - How does FastAPI's approach to authentication compare to frameworks you've used before? - What advantages does FastAPI's dependency injection system provide for authentication? - How does type hinting in FastAPI make security implementation clearer compared to other frameworks? - What patterns from other frameworks can you identify in the JWT implementation?

Part 4: Mental Model Translation Exercise

Task: Create a conceptual diagram or table that maps your existing mental model of web frameworks to FastAPI concepts. Approach: list the key components you understand from familiar frameworks (routes, middleware, controllers, etc.); ask AI to help you map these to FastAPI equivalents; note the differences in approach and philosophy; update your mental model to incorporate FastAPI's unique aspects.

Example prompt: "If I think of Django's middleware, views, and models as the core architectural components, what would be the corresponding mental model for FastAPI applications?"

Submission for Exercise 18 — Contextual Learning with FastAPI:

1. Your personal "translation table" mapping concepts from frameworks you know to their FastAPI equivalents (Part 1).

2. A short summary of FastAPI's design philosophy and how it should influence how you structure applications (Part 2).
3. Your JWT-token authentication implementation, plus answers to the reflection questions on how it compares to other frameworks (Part 3).
4. Your conceptual diagram/table mapping your existing web-framework mental model to FastAPI concepts (Part 4).

Suggested prompt *(for Exercise 18 — Part 1, "Framework Comparison"):*

> I already know [Flask / Django] and I want to learn FastAPI by mapping it onto what I already understand. Please build me a "translation table": (1) compare FastAPI and Flask — what concepts are similar and what's fundamentally different; (2) map Flask Blueprints / Django middleware and views to their FastAPI equivalents (routers, dependency injection); (3) contrast FastAPI's Pydantic request validation with Django form validation. Then explain the *why* behind FastAPI's design (Pydantic, automatic docs, heavy use of type hints, async-first), so I can update my mental model. Where useful, show short side-by-side snippets.

Exercise 19 — Documentation Navigation for FastAPI

In this exercise, you'll practice using AI to navigate and extract practical value from FastAPI's documentation.

Part 1: Documentation Summarization

Task 1: Use AI to create a learning roadmap based on the FastAPI documentation. Prompts to try: - "Based on the FastAPI documentation at https://fastapi.tiangolo.com/, what would be an effective reading order for someone new to the framework?" - "What are the 5 most important sections in FastAPI's documentation for a developer who needs to build a REST API quickly?" - "Summarize the key points from FastAPI's dependency injection documentation in a way that highlights practical applications."

Goal: Create a personalized reading guide for the FastAPI documentation that matches your learning style and project needs.

Part 2: Documentation Deep Dive

Task 2: Select a complex FastAPI feature you want to understand better (e.g., dependency injection, background tasks, or WebSockets). Prompts to try: - "I'm looking at FastAPI's WebSocket documentation at https://fastapi.tiangolo.com/advanced/websockets/. Can you explain the core concepts and why they matter?" - "What parts of FastAPI's security documentation are most relevant for building an API that requires authentication?" - "I'm confused by the Depends function in FastAPI. Can you explain its purpose and when I should use it versus when I shouldn't?"

Goal: Create notes that extract the essential information from a complex documentation section, focusing on practical application.

Part 3: Concept to Code Translation

Task 3: Use AI to translate abstract FastAPI concepts into practical code examples. (The course page shows an example response covering dependency injection, Pydantic request/response models, background tasks, path operation parameters, and exception handling.) Prompts to try: - "Based on FastAPI's documentation, show me practical code examples of dependency injection patterns for different scenarios." - "The FastAPI docs mention 'path operation decorators' - can you explain what these are and show examples of different decorators and when to use each one?" - "How do FastAPI's background tasks work according to the documentation? Show me example code for sending emails in the background after an API response."

Goal: Create a reference guide that connects abstract FastAPI concepts from the documentation with concrete code implementations.

Part 4: Comprehensive Documentation Challenge

Task 4: Use AI to help you build a complete mini-application based primarily on FastAPI documentation.

Challenge: Create a FastAPI application that implements a RESTful API for a simple blog with the following features: - User registration and authentication - CRUD operations for blog posts - Comments on blog posts - Basic search functionality

Approach: ask AI to identify the relevant documentation sections for each feature; request summaries of key concepts for each component; ask for practical examples based on the documentation; combine these examples into a cohesive application; have AI explain how the documentation informed the implementation choices.

Example prompt: "I'm building a blog API with FastAPI. Based on the official documentation, which sections should I focus on for implementing user authentication, and can you provide a practical example based strictly on the approach recommended in the docs?"

Challenge yourself to apply what you learned, and remember to prompt in increments, focussing on one learning topic or feature at a time.

Submission for Exercise 19 — Documentation Navigation for FastAPI:

1. Your personalized FastAPI documentation reading guide (Part 1).
2. Your deep-dive notes extracting the essentials of one complex feature, e.g. dependency injection, background tasks, or WebSockets (Part 2).
3. Your reference guide connecting abstract FastAPI concepts to concrete code examples (Part 3).
4. The blog mini-application (user auth, CRUD for posts, comments, basic search) built primarily from the documentation, with notes on how the docs informed your implementation choices (Part 4).

Suggested prompt *(for Exercise 19 — Part 1 and Part 4):*

> I want to learn FastAPI efficiently from its official documentation (https://fastapi.tiangolo.com/) and I'd like your help navigating it. First: (1) suggest an effective

> reading order for a newcomer; (2) name the 5 most important sections for building a REST API quickly; (3) summarize the dependency-injection docs with a focus on practical use. Then help me build a simple blog API incrementally — for each feature (user registration and authentication, CRUD for blog posts, comments, basic search), point me to the relevant docs sections, summarize the key concepts, and give a practical example based strictly on the approach the docs recommend. Let's do one feature at a time.

Exercise 20 — Understanding FastAPI Code Patterns

In this exercise, you'll practice using AI to understand complex code patterns in FastAPI, focusing on the dependency injection system and middleware concepts.

Part 1: Analyzing Complex Code

Task: Use AI to help you understand a FastAPI code excerpt (provided on the course page) that uses several advanced patterns — a generic `Repository[T]` pattern, a `UserRepository`, a `get_db` session dependency, a `UserService` layer, a `TimingMiddleware`, JWT-based `get_current_user`, a `requires_role` decorator, an `asynccontextmanager` lifespan handler, and admin endpoints. Prompts to try: - "Can you explain the Repository pattern being used in this FastAPI code? Why would someone structure their code this way?" - "What is the purpose of the Generic[T] in the Repository class, and how does it make the code more maintainable?" - "How does dependency injection work in this FastAPI application? Can you explain each layer of dependencies?" - "Can you break down the role-based access control system in this code and explain how it works?"

Goal: Create a clear understanding of the design patterns used in this code and why they might be beneficial.

Part 2: Tracing Execution Flow

Task: Use AI to trace the execution flow when a request is made to the `/admin/users/` endpoint. Prompts to try: - "Can you trace what happens when a request is made to the '/admin/users/' endpoint from start to finish?" - "What are all the middleware, dependencies, and function calls that occur during a request to the '/admin/users/' endpoint?" - "How does the authentication and authorization process work in this code when accessing the admin endpoint?"

Goal: Create a step-by-step flowchart or sequence diagram of the request handling process.

Part 3: Simplifying Complex Concepts

Task: Have AI break down some of the more advanced concepts in the code. Prompts to try: - "Can you explain the asynccontextmanager decorator and the lifespan function in simpler terms? What problem does it solve?" - "How does the TimingMiddleware work, and what would be a simple example of using it?" - "Can you explain the JWT authentication flow in this application in a simplified way that a junior developer would understand?"

Goal: Create a "translation guide" that explains advanced patterns in simpler terms.

Part 4: Building Understanding Through Implementation

Task: Use your new understanding to implement a new feature in this codebase.

Challenge: Implement a simple logging system that records user actions (like logins and admin actions) to a database, using the same patterns already present in the code.

Approach: start by asking AI to explain the existing patterns in detail; have AI guide you on how to extend the codebase with the new feature; implement the feature using the same architectural approaches; ask AI to review your implementation and suggest improvements.

Reflection Questions: - How does implementing this feature help you understand the overall architecture? - Which design patterns did you find most useful in the original code? - How would you explain the repository pattern and dependency injection to a colleague? - How did tracing the execution flow help you understand where to add your code?

Submission for Exercise 20 — Understanding FastAPI Code Patterns:

1. A clear explanation of the design patterns used in the code and why they're beneficial (Part 1).
2. A step-by-step flowchart or sequence diagram of the request-handling process for the `/admin/users/` endpoint (Part 2).
3. A "translation guide" explaining the advanced concepts (lifespan/ asynccontextmanager, TimingMiddleware, JWT flow) in simpler terms (Part 3).
4. Your implemented logging feature (recording user actions to a database using the existing patterns), plus answers to the reflection questions (Part 4).

Suggested prompt *(for Exercise 20 — Part 1 and Part 2):*

> I'm trying to understand an advanced FastAPI codebase that uses several patterns at once. Here's the code: `python [Paste the advanced FastAPI excerpt with Repository[T], UserService, TimingMiddleware, get_current_user, requires_role, and lifespan]` Please help me decode it: (1) explain the generic Repository pattern and what `Generic[T]` buys us; (2) walk through each layer of dependency injection (`get_db` → repository → service → `get_current_user`); (3) explain how the `requires_role("admin")` role-based access control works; (4) trace, step by step, everything that happens — middleware, dependencies, and function calls — when a request hits `/admin/users/` , from authentication through authorization to the response. Then guide me in adding a logging system that records user actions using the same patterns, and review my implementation.

Summary

#	Chapter	Exercise
1	Comprehend existing codebases	Knowing where to start
2		Codebase Exploration Challenge

#	Chapter	Exercise
	Comprehend existing codebases	
3	Comprehend existing codebases	Algorithm Deconstruction Challenge
4	Documentation with AI	Code Documentation
5	Documentation with AI	API Documentation
6	Documentation with AI	README and User Guide Documentation
7	Debug errors	Error Diagnosis Challenge
8	Debug errors	Performance Optimization Challenge
9	Debug errors	AI Solution Verification Challenge
10	Help with testing	Using AI to help with testing
11	Refactor and enhance code	Understanding What to Change with AI
12	Refactor and enhance code	Function Decomposition Challenge
13	Refactor and enhance code	Code Readability Challenge
14	Refactor and enhance code	Design Pattern Implementation Challenge
15	Current programming language	Applying AI to deepen programming language understanding
16	New programming language	Learning a New Programming Language with AI
17	New framework/library/API	Getting Started with FastAPI
18	New framework/library/API	Contextual Learning with FastAPI
19	New framework/library/API	Documentation Navigation for FastAPI
20	New framework/library/API	Understanding FastAPI Code Patterns

Source: WeThinkCode_ "Coding & Learning: Using GenAI To Support Software Development" — https://ai.wethinkco.de/ai-software/ (accessed via the ai.wethinkcode.co.za account).